PHY F376: A study of hybrid classical-quantum algorithms

Week 1 (20/8/2026) - Sample-Based Quantum Diagonalization: A Study of the N2 Example

Khushi Pandey (2023B5A70951G)

Motivation

Finding a molecule's ground-state energy means diagonalizing its electronic Hamiltonian, and this matrix gets exponentially larger as more orbitals are included, so exact classical diagonalization quickly becomes infeasible. Sample-based quantum diagonalization (SQD) works around this by splitting the task between quantum and classical hardware: a quantum processor is used only to generate samples of likely electron configurations, and a classical computer then diagonalizes the Hamiltonian within the much smaller subspace spanned by those samples. This division of labour was the main idea I took away from both resources this week.

The Four-Step Workflow

Both the tutorial and the function-template guide follow the same four-step Qiskit pattern: map the problem, optimize the circuit, execute it, and post-process the results. In the N2 example, Step 1 defines the molecule in PySCF and picks an active space (8 orbitals with 5 alpha and 5 beta electrons, after freezing 2 core orbitals in the small-scale case), and also runs a classical CCSD calculation just to get the t2 amplitudes needed to initialize the quantum circuit. Step 2 builds the LUCJ (local unitary cluster Jastrow) circuit using ffsim on a Hartree-Fock reference state, then transpiles it for the target device. Step 3 samples this circuit many times, giving bitstrings that each represent one possible arrangement of electrons among the orbitals. Step 4 is where SQD itself comes in: bitstrings that break particle-number conservation are corrected, the valid ones are grouped into random batches, the Hamiltonian is diagonalized inside each batch, and the orbital-occupancy estimate is updated every round. This loop is called self-consistent configuration recovery, and it's really the core idea behind SQD.

Results of the N2 Example

The tutorial's noiseless simulator run (STO-6G basis) converged almost immediately, since there was no noise to correct for. The hardware run, on the larger cc-pVDZ basis set (26 orbitals), was more revealing: only about 2% of sampled bitstrings were valid, but that is still roughly 20,000 times better than the $\sim$0.0001% expected from pure random sampling. This suggested that the circuit really is capturing useful chemical information, just with a lot of noise mixed in. Over five recovery iterations, the energy estimate improved steadily from about -109.14 Ha to -109.19 Ha, ending up around 40 mHa from the reference value. This is short of chemical accuracy ($\sim$1.6 mHa), which matches what the guide says about larger, noisier systems needing more shots, batches, or iterations to get there.

Relation to the Chemistry Function Template

The function-template guide packages this same pipeline into a reusable service that can be deployed on Qiskit Serverless, configured through a molecule specification and a few user-set options (shot count, number of batches, samples per batch, recovery iterations). It also adds an implicit solvent model (IEF-PCM), giving a solvation free energy alongside the SQD ground-state energy, which wasn't needed for the gas-phase N2 example.

Concluding Remarks

This week helped me understand why quantum sampling and classical diagonalization are combined the way they are, and how configuration recovery pulls a usable signal out of noisy hardware data. Since this week was mostly spent reading and understanding these ideas, this report does not contain any new results beyond what is already in the two sources. With this foundation in place, I hope to begin working with an actual Hamiltonian soon.

Sources:

- IBM Quantum, "Sample-based quantum diagonalization of a chemistry Hamiltonian" (N2 tutorial)
- IBM Quantum, "Build a Qiskit Function for chemistry simulation"